

Coding Exercise — Intel Messages: DAL Class (14:45 – 16:30)

Scenario: Over the past two days, the intel messages system grew from a setup script to a CRUD API — but the code is spread across multiple files and functions with no shared structure. Today you refactor and extend it: everything that touches `intel_messages` moves into a single **Data Access Layer class**. The class owns the connection, the table setup, the full CRUD, and the query methods. The FastAPI server becomes thin — it calls the DAL and returns what it gets back.

Continuity note: Same container , same database, same `intel_messages` table. Do **not** recreate anything — verify the table exists before you start.

Before writing any code — planning phase (15 minutes)

Answer in writing:

1. What is a Data Access Layer? What responsibility does it have — and what does it explicitly **not** do?
2. In the procedural approach from the past two days, every function called `get_connection()` on its own. What problem does that create? How does moving the connection into a class constructor solve it?
3. The class will have a method called `setup()` . What should it do — and why is it a method on the class rather than a separate file?
4. Look at the class skeleton below. The constructor receives `host` , `user` , `password` , and `database` . Why is it better to pass these in than to hardcode them inside the class?
5. You already wrote `create_message` , `update_message` , and `delete_message` as standalone functions. What changes when you move them into a class — specifically regarding `self` , the connection, and return values?

Step 1 — Verify the table and seed data

Start the container if needed:

```
docker start mysql-w7
```

Check that `intel_messages` has data. If the table is empty or has fewer than 6 rows, insert the seed rows:

```
docker exec -it mysql-w7 mysql -uroot -proot -e "
USE soldiers_db;
INSERT INTO intel_messages (unit, classification, content, source) VALUES
  ('8200', 'confidential', 'Suspicious movement detected near northern
grid.', 'field agent'),
  ('8200', 'secret', 'Encrypted signal intercepted on frequency 312.',
'sigint'),
  ('9900', 'top_secret', 'Satellite image shows vehicle convoy at dawn.',
'satellite'),
  ('9900', 'unclassified', 'Routine patrol completed. No incidents
reported.', NULL),
  ('8200', 'secret', 'Drone feed shows activity near the eastern
border.', 'drone feed'),
  ('Unit3', 'confidential', 'Local source reports increased foot traffic.',
NULL);
SELECT * FROM intel_messages;
"
```

Inspect the schema before writing anything:

```
docker exec -it mysql-w7 mysql -uroot -proot -e "USE soldiers_db; DESCRIBE
intel_messages;"
```

Before you run: Write down every column name and type you expect to see — then compare.

Step 2 — Write `intel_messages_dal.py`

Create a new file named `intel_messages_dal.py`.

The class skeleton

Your file must define exactly this class. Do **not** change method names or signatures. Fill in every method body yourself.

```
import mysql.connector

class IntelMessagesDAL:
```

```

VALID_CLASSIFICATIONS = ('unclassified', 'confidential', 'secret',
'top_secret')

def __init__(self, host: str, user: str, password: str, database: str,
logger: Logger):
    # Store connection parameters on self
    # store logger object reference in self

    ...

# ----- setup

def get_conn(self):

    #create connection from params stored in self
    #store the connection in self
    #return conn

def setup(self) -> None:
    # Create the intel_messages table if it does not exist
    # Column definitions: id, unit, classification (ENUM), content,
source, created_at
    # Commit after execution
    ...

# -----
schema

def get_schema(self) -> list[dict]:
    # Query INFORMATION_SCHEMA.COLUMNS for the intel_messages table
    # Return a list of dicts: [{"column": ..., "type": ...}, ...]
    ...

# ----- read (all)

def get_all(self) -> list[dict]:
    # Return every row in intel_messages as a list of dicts
    ...

# ----- read (by id)

def get_by_id(self, message_id: int) -> dict | None:
    # Return the single row where id = message_id, or None if not found

```

```

...

# ----- create

def create(self, unit: str, classification: str, content: str, source: str
| None) -> int:
    # Insert a new row (do NOT pass created_at - let MySQL set it)
    # Commit the transaction
    # Return the auto-generated id (lastrowid)
    ...

# ----- update

def update(self, message_id: int, data: dict) -> bool:
    # Build a dynamic SET clause from the keys in data
    # Only update the columns that are present in data
    # Commit the transaction
    # Return True if a row was changed, False if the id did not exist
    # Never use f-strings for values - only %s
    ...

# ----- delete

def delete(self, message_id: int) -> bool:
    # Delete the row where id = message_id
    # Commit the transaction
    # Return True if a row was deleted, False if the id did not exist
    ...

# ----- queries

def get_by_unit(self, unit: str) -> list[dict]:
    # All messages where unit matches, ordered by created_at DESC
    ...

def get_by_classification(self, classification: str) -> list[dict]:
    # All messages at the given classification level
    ...

def get_by_unit_and_classification(self, unit: str, classification: str) -
> list[dict]:
    # Both filters combined with AND
    ...

def get_distinct_units(self) -> list[str]:
    # All unique unit values - return a plain list of strings, not dicts

```

```

...

def search_content(self, term: str) -> list[dict]:
    # Rows where content contains term (partial match)
    ...

def get_missing_source(self) -> list[dict]:
    # Rows where source IS NULL
    ...

# ----- close

def close(self) -> None:
    # Close the cursor and the connection
    ...

```

Implementation rules

- The connection and cursor are opened **once** in `__init__` and reused by all methods
- Use `cursor(dictionary=True)` so every row comes back as a dict
- Use `%s` for all values — never build the value part of a query with an f-string or string concatenation
- Column **names** may appear in f-strings (e.g. for the dynamic `SET` clause in `update`) — values may not
- Every write method (`create`, `update`, `delete`, `setup`) must call `self.connection.commit()`
- `get_distinct_units` should return `list[str]`, not `list[dict]` — extract just the unit value from each row

Comprehension questions (answer in writing before moving to Step 3)

1. `update` builds the `SET` clause dynamically. Given `data = {"classification": "secret", "content": "new text"}`, write out exactly what the final SQL string and the values tuple look like after your loop runs.
 2. `get_distinct_units` returns `list[str]`. What one line of Python turns a list of single-key dicts into a flat list of strings?
 3. Why does `close()` need to close the cursor **before** the connection — what happens if you close the connection first?
-

Step 3 — Write `main_messages_v3.py`

Create a new file `main_messages_v3.py` .

The server must:

- Instantiate `IntelMessagesDAL` once at module level (outside any route function)
- Call `dal.setup()` once on startup — use FastAPI's `lifespan` or a startup event
- Call `dal.close()` on shutdown
- Run on **port 8002**

Routes

Route	Method	Query params / path params	Success	Not found
<code>/schema</code>	GET	—	200	—
<code>/messages</code>	GET	<code>unit</code> , <code>classification</code> (both optional)	200	—
<code>/messages/units</code>	GET	—	200	—
<code>/messages/search</code>	GET	<code>q</code> (required)	200	400 if missing/empty
<code>/messages/missing-source</code>	GET	—	200	—
<code>/messages</code>	POST	—	201	—
<code>/messages/{message_id}</code>	GET	—	200	404
<code>/messages/{message_id}</code>	PUT	—	200	404
<code>/messages/{message_id}</code>	DELETE	—	200	404

Detail: GET `/messages`

Handle all four filter combinations with a single route:

unit present	classification present	Calls
No	No	<code>dal.get_all()</code>
Yes	No	<code>dal.get_by_unit(unit)</code>
No	Yes	<code>dal.get_by_classification(classification)</code>

unit present	classification present	Calls
Yes	Yes	<code>dal.get_by_unit_and_classification(unit, classification)</code>

Detail: GET /messages/search

- Return `400 Bad Request` with a descriptive message if `q` is missing or an empty string
- Otherwise return the results of `dal.search_content(q)`

Detail: GET /messages/missing-source

- Return `200` with an empty list if no rows match — **not** `404`

Detail: POST body

```
{
  "unit": "8200",
  "classification": "confidential",
  "content": "Target vehicle spotted near grid 7.",
  "source": "drone feed"
}
```

`source` is optional.

Detail: PUT body

Accept any subset of the writable columns:

```
{
  "classification": "top_secret"
}
```

Only the fields present in the body should be updated.

Design question (answer in writing): The DAL is instantiated once and its connection stays open for the entire server lifetime. What is the advantage of this over opening a new connection per request? What is one potential risk?

Step 4 — Test your server

```
uvicorn main_messages_v3:app --reload --port 8002
```

Test in this exact order:

```
1. POST /messages      body: {"unit": "8200", "classification":  
"confidential", "content": "Target vehicle spotted near grid 7.", "source":  
"drone feed"}  
2. POST /messages      body: {"unit": "9900", "classification": "secret",  
"content": "Encrypted signal on frequency 312.", "source": "sigint"}  
3. GET  /messages      → all messages  
4. GET  /messages?unit=8200      → only unit 8200  
5. GET  /messages?classification=secret      → only secret  
6. GET  /messages?unit=8200&classification=secret      → both filters  
7. GET  /messages/units      → distinct unit list  
8. GET  /messages/search?q=movement      → partial content match  
9. GET  /messages/search      → should return 400  
10. GET /messages/missing-source      → rows without source  
11. GET  /messages/1      → single message  
12. PUT  /messages/1      body: {"classification": "top_secret"}  
13. GET  /messages/1      → classification is now  
top_secret  
14. DELETE /messages/2  
15. GET  /messages      → message 2 is gone  
16. GET  /messages/99      → 404
```

After step 14, confirm directly in MySQL:

```
docker exec -it mysql-w7 mysql -uroot -proot -e "USE soldiers_db; SELECT *  
FROM intel_messages;"
```

Extension (for students who finish early)

Extension 1 — Sort direction:

- Add an optional `sort` query param to `GET /messages` (`asc` or `desc`, default `desc`)
- Return `400` if the value is anything other than those two strings
- The sort direction must affect the SQL `ORDER BY` — but you cannot use `%s` for it. How do you inject it safely?

Extension 2 — Classification validation in the DAL:

- In `create` and `update`, validate the `classification` value against `VALID_CLASSIFICATIONS` before running any SQL
- Raise a `ValueError` with a clear message if it is invalid
- In the FastAPI route, catch that `ValueError` and return `422 Unprocessable Entity`

Extension 3 — Reconnect on failure:

- What happens if the MySQL connection drops while the server is running?
- Add a `_ensure_connection()` private method that checks `self.connection.is_connected()` and reconnects if needed
- Call it at the top of every public method

Extension 4 — Research:

- What is the difference between `PUT` and `PATCH` in HTTP semantics? Which verb more accurately describes what your `PUT /messages/{id}` route actually does — and why?
- What is a SQL `INDEX`? Why might adding one to the `unit` column of `intel_messages` matter as the table grows?
- What does `ON DELETE CASCADE` mean on a foreign key? If `intel_messages` had a foreign key to `soldiers`, what would happen to the messages when a soldier is deleted?